Лабораторна робота № 2

Паралельні цикли в OpenMP програмах

Теоретичні відомості

Як вже зазначалося раніше, програмний код блоку директиви **parallel** за замовчуванням виконується усіма потоками. Даний спосіб може бути корисний, коли потрібно виконати одні й ті ж дії багаторазово. або коли один і той же програмний код може бути застосований для виконання обробки різних даних. Останній варіант досить широко використовується при розробці паралельних алгоритмів і програм і зазвичай іменується розпаралелюванням за даними. У рамках даного підходу в OpenMP поряд із звичайним повторенням в потоках одного і того ж програмного коду, як у директиві **parallel**, можна здійснити поділ ітеративно. Така можливість є тим більш важливою, оскільки в багатьох випадках саме в циклах виконується основна частина трудомістких обчислень. Якщо в паралельній області зустрівся оператор циклу, то, відповідно до загального правила, він буде виконаний усіма потоками поточної групи, тобто кожен потік виконає всі ітерації даного циклу. Для розпаралелювання циклів в OpenMP застосовується директива **for**:

#pragma omp for [опція [[,] опція]...]

Ця директива відноситься до блоку, що йде слідом за даною директивою, котрий містить оператор **for**.

Слід зазначити, що для розпаралелювання цикл **for** повинен мати деякий «канонічний»[1] тип циклу з лічильником:

for (**index** = **first**; **index** < **end**; **increment_expr**)

Тут **index** повинен бути цілочисельною змінною; на місці знака < у виразі для перевірки закінчення циклу може знаходитися будь-яка операція порівняння <=, > або > =. Операція зміни ітератора циклу повинна мати одну з наступних форм:

- **index**++, ++**index**,
- **index**−−, −−**index**,
- **index**+=**incr**, **index**−=**incr**,
- **index**=**index**+**incr**, **index**=**incr**+**index**,
- **index**=**index**−**incr**.

І, звичайно ж, змінні, котрі використовуються в заголовку оператора циклу, не повинні змінюватися в тілі циклу, тобто між ітераціями циклу немає інформаційної залежності.

Можливі опції:

- **private**(**список**) – задає список змінних, для яких створюється локальна копія в кожному потоці; початкове значення локальних копій змінних зі списку не визначене;
- **firstprivate** (**список**) – теж що і **private**; початкове значення локальних копій змінних зі списку задається значеннями цих змінних в основному потоці;
- **lastprivate** (**список**) – змінним, перерахованим у списку, присвоюється результат, отриманий в останній ітерації циклу;

[1] Сенс вимоги «канонічності» полягає в тому, щоб на момент початку виконання циклу існувала можливість визначення числа ітерацій циклу

- **reduction**(**оператор**: **список**) – задає оператор і список спільних змінних; для кожної змінної створюються локальні копії в кожному потоці; локальні копії ініціалізуются відповідно до типу оператора (для адитивних операцій – 0 або його аналоги, для мультиплікативних операцій – 1 або її аналоги); над локальними копіями змінних після виконання всіх операторів паралельної області виконується заданий оператор; оператори для мови Сі – +, −,∗,&,|,^,&&,||,**max**,**min**; порядок виконання операторів не визначений, тому результат може відрізнятися від запуску до запуску;
- **schedule**(**type** [, **chunk**]) – опція задає, яким чином ітерації циклу розподіляються між потоками;
- **collapse**(**n**) – вказує на те, що **n** щільно вкладених послідовних циклів асоціюється з даною директивою; для циклів утворюється загальний простір ітерацій, який ділиться між потоками; якщо опція **collapse** не задана, то директива стосується тільки одного циклу, розташованого за директивою **for**;
- **ordered** – опція, що говорить про те, що в циклі можуть зустрічатися директиви **ordered**; в цьому випадку визначається блок всередині тіла циклу, який повинен виконуватися в тому порядку, в якому ітерації йдуть в послідовному циклі;
- **nowait** – після виконання виділеної ділянки відбувається неявна бар'єрна синхронізація паралельно працюючих потоків: їх подальше виконання відбувається тільки тоді, коли всі вони досягнуть даної точки; якщо в подібній затримці немає необхідності, опція **nowait** дозволяє потокам, котрі вже дійшли до кінця ділянки, продовжити виконання без синхронізації з іншими.

Якщо в блоці директиви **parallel** немає нічого, крім директиви **for**, то обидві директиви можна об'єднати в одну, як у прикладі 2.1.

Приклад 2.1: Розрахунок числа π.

```
//
#include <stdio.h>
double f(double y) {return(4.0/(1.0+y*y));}
int main()
{
  double w, x, sum, pi;
  int i;
  int n = 1000000;
  w = 1.0/n;
  sum = 0.0;
  #pragma omp parallel for private(x) shared(w) reduction(+:sum)
  for(i=0; i < n; i++)
  {
    x = w*(i-0.5);
    sum = sum + f(x);
  }
  pi = w*sum;
  printf("pi = %f\n", pi);
  return 0;
}
```

Управління розподілом ітерацій циклу між потоками

При різному обсязі обчислень в різних ітераціях циклу бажано мати можливість керувати розподілом ітерацій циклу між потоками – в OpenMP це забезпечується за допомогою параметра **schedule** директиви **for**. Поле **type** опції **schedule** може приймати наступні значення:

- **static** – статичний спосіб розподілу ітерацій до початку виконання циклу. Якщо поле **chunk** не вказано, то ітерації діляться порівну між потоками. При заданому зна-

ченні **chunk** ітерації циклу діляться на блоки розміру **chunk** і ці блоки розподіляються між потоками до початку виконання циклу.

- **dynamic** – динамічний спосіб розподілу ітерацій. До початку виконання циклу потокам виділяються блоки ітерацій розміру **chunk** (якщо поле **chunk** не вказано, то вважається **chunk** = 1). Подальше виділення ітерацій (також блоками розміру **chunk**) здійснюється в момент завершення потоками своїх раніше призначених ітерацій.
- **guided** – динамічний розподіл ітерацій, при якому розмір блоку зменшується з деякого початкового значення до величини **chunk** (за замовчуванням **chunk** = 1) пропорційно кількості ще не розподілених ітерацій, поділеному на кількість потоків, що виконують цикл. Розмір початкового блоку залежить від реалізації. У ряді випадків такий розподіл дозволяє акуратніше розділити роботу і збалансувати завантаження потоків. Кількість ітерацій в останньому блоці може виявитися менше значення **chunk**.
- **auto** – спосіб розподілу ітерацій вибирається компілятором і/або системою виконання. Параметр **chunk** при цьому не задається.
- **runtime** – спосіб розподілу ітерацій вибирається під час роботи програми за значенням змінної середовища **OMP_SCHEDULE**. Параметр **chunk** при цьому не задається. Так, наприклад, для завдання динамічного способу при розмірі блоку ітерацій 3, слід визначити:
  setenv OMP_SCHEDULE "dynamic,3".
  Змінити значення змінної **OMP_SCHEDULE** з програми можна за допомогою виклику функції **omp_set_schedule()**.

Приклад 2.2 демонструє використання опції **schedule** з різними параметрами. В паралельній області виконується цикл, ітерації якого будуть розподілені між існуючими потоками. На кожній ітерації буде надруковано, який потік виконав дану ітерацію. В тіло циклу вставлена затримка, що імітує деякі обчислення.

Приклад 2.2: Використання опції **schedule**.

```
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[])
{
  int i;
  #pragma omp parallel private(i)
  {
    #pragma omp for schedule(static)
    // schedule(static, 1)
    // schedule(static, 2)
    // schedule(dynamic)
    // schedule(dynamic, 2)
    // schedule(guided)
    // schedule(guided, 2)
    for (i=0; i<10; i++)
    {
      printf("Потік %d виконав ітерацію %d\n",
             omp_get_thread_num(), i);
      sleep(1);
    }
  }
}
```

Управління порядком виконання обчислень

У результаті розпаралелювання циклу порядок виконання ітерацій не фіксований: залежно від стану середовища черговість виконання ітерацій може змінюватися. Якщо ж

для ряду дій в циклі необхідно зберегти первинний порядок обчислень, який відповідає послідовному виконанню ітерацій в послідовній програмі, то бажаного результату можна досягти за допомогою директиви **ordered** (при цьому для директиви **for** повинен бути вказаний параметр **ordered**). Пояснимо сказане на фрагменті програми друку сум елементів рядків матриці. Розрахунок відбуватиметься в деякому довільному порядку, при необхідності друку по порядку розташування рядків слід додати директиву **#pragma omp ordered**.

```
#pragma omp parallel for shared(a) private(i,j,sum) \
schedule (dynamic, chunk) ordered
{
  for (i=0; i < NMAX; i++) {
    sum = 0;
    for (j=i; j < NMAX; j++)
      sum += a[i][j];
    #pragma omp ordered
    printf ("Сума елементів рядка %d рівна %f\n",i,sum);
  }
}
```

Пояснимо додатково, що параметр **ordered** керує порядком виконання тільки тих дій, які виділені директивою **ordered** – виконання решти дій, що залишилися в ітераціях циклу надалі може відбуватися паралельно. Важливо пам'ятати також, що директива **ordered** може бути застосована в тілі циклу тільки один раз.

Слід зазначити, що необхідність збереження порядку обчислень може призвести до затримок при паралельному виконанні ітерацій, що обмежить можливість отримання максимального прискорення обчислень.

Функції для заміру часу

У OpenMP передбачені функції для роботи з системним таймером.

Функція **double omp_get_wtime()** повертає, в потоці котрий її викликав, астрономічний час в секундах, що минув з деякого моменту в минулому. Якщо деяку ділянку програми оточити викликами даної функції, то різниця повернених значень покаже час роботи даної ділянки. Гарантується, що момент часу, використовуваний в якості точки відліку, не буде змінено за час існування процесу. Таймери різних потоків можуть бути не синхронізовані і видавати різні значення.

Функція **double omp_get_wtick()** повертає роздільну здатність таймера в секундах. Цей час можна розглядати як міру точності таймера.

Приклад 2.3 ілюструє застосування функцій **omp_get_wtime()** та **omp_get_wtick()** для роботи з таймерами в OpenMP. У даному прикладі проводиться вимір початкового часу, потім відразу замір кінцевого часу. Різниця часів дає час на замір часу, також вимірюється точність системного таймера.

Приклад 2.3: Робота з системними таймерами.

```
#include <stdio.h>
#include <omp.h>
int main(int argc, char *argv[])
{
  double start_time, end_time, tick;
  start_time = omp_get_wtime();
  end_time = omp_get_wtime();
  tick = omp_get_wtick();
  printf("Час на замір часу %lf\n", end_time-start_time);
  printf("Точність таймера %lf\n", tick);
}
```

Хід роботи

1. Ознайомитись з директивою **#pragma omp for**, та її опціцями.
2. Написати програму згідно індивідуального завдання. За допомогою функції **omp_get_wtime()** заміряйте час роботи програми за різної кількості потоків та розміру вхідних даних.
   Вказівки: Використати опцію **reduction**.
3. Використовуючи опцію **schedule** (з різними параметрами) модифікуйте прграму таким чином, щоб на екран виводилось повідоилення про те, який потік, яку ітерацію виконує.
   [<**Номер потоку**>]: **calculation of the iteration number** <**Номер ітерації**>.
4. Оформити звіт про виконання лабораторної роботи.

Індивідуальні завдання

Напишіть паралельну OpenMP програму котра:

1. реалізує пошук максимального значення вектора;
2. знаходить суму елементів матриці;
3. знаходить суму максимальних елементів рядків матриці;
4. знаходить найменше значення суми елементів рядків матриці;
5. знаходить максимальне значення суми елементів стовпців матриці;
6. реалізує додавання квадратних матриць;
7. знаходить суму найменших елементів стовпців матриці;
8. реалізує множення квадратних матриць;
9. знаходить максимальний елемент матриці;
10. знаходить максимальне значення суми елементів рядків матриці;
11. знаходить суму найменших елементів рядків матриці;
12. знаходить найменше значення суми елементів стовпців матриці;
13. реалізує скалярний добуток двох векторів;
14. знаходить суму максимальних елементів стовпців матриці;